29

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

수명주기 훅

제2부 NestJS 핵심 · 7장 수명주기와 플랫폼

서버가 뜰 때와 내려갈 때 Nest가 부르는 메서드



© 한상훈 NestJS 백엔드 마스터

문자를 모아서 보내는 SmsOutbox

sms-outbox.ts

```ts
@Injectable()
export class SmsOutbox {
  private readonly queue: Sms[] = [];
  private from: string;

  constructor() {
    this.loadSender();
    setInterval(() => this.flush(), 10_000);
  }

  private async loadSender() {
    const res = await fetch(`${this.baseUrl}/sender`);
    ({ from: this.from } = await res.json());
  }

  enqueue(to: string, text: string) {
    this.queue.push({ from: this.from, to, text });
  }

  async flush() { /* 큐를 비워 발송사 /send로 한 번에 */ }
}
```

→ SmsSender.send가 문장을 만들어 돌려주면서 큐에 넣음

→ 발신번호: 발송사에 미리 등록한 번호, 부팅 때 받아 옴

→ 10초마다 모아서 한 번에 발송 (발송사 호출 횟수 절약, 가정)

© 한상훈 NestJS 백엔드 마스터

배포 순간의 예매 두 건

요청 쪽 기록

```
14:48:48.627 R1 요청 보냄
14:48:50.629 R2 요청 보냄
14:48:51.325 R1 끝 [201]
  {"id":1,"seatId":"A-3-15",...,"message":"[문자 → 010-1111-2222] A-3-15 예매가 확정되었습니다"}
14:48:52.632 kill -TERM 서버
14:48:52.650 R2 끝
  curl: (52) Empty reply from server
```

→ 결제사 응답에 2.5초 걸리게 한 가짜 결제사

→ `kill -TERM`: 배포 도구가 옛 프로세스에 보내는 종료 신호와 같은 신호

© 한상훈 NestJS 백엔드 마스터

결제사와 문자 발송사 쪽 기록

가짜 외부 서버 로그

```
14:48:51.313 [결제사] 승인 pg-50000
14:48:53.137 [결제사] 승인 pg-60000
```

R1

응답은 201

큐에 넣은 확정 문자는 발송 기록 없음

R2

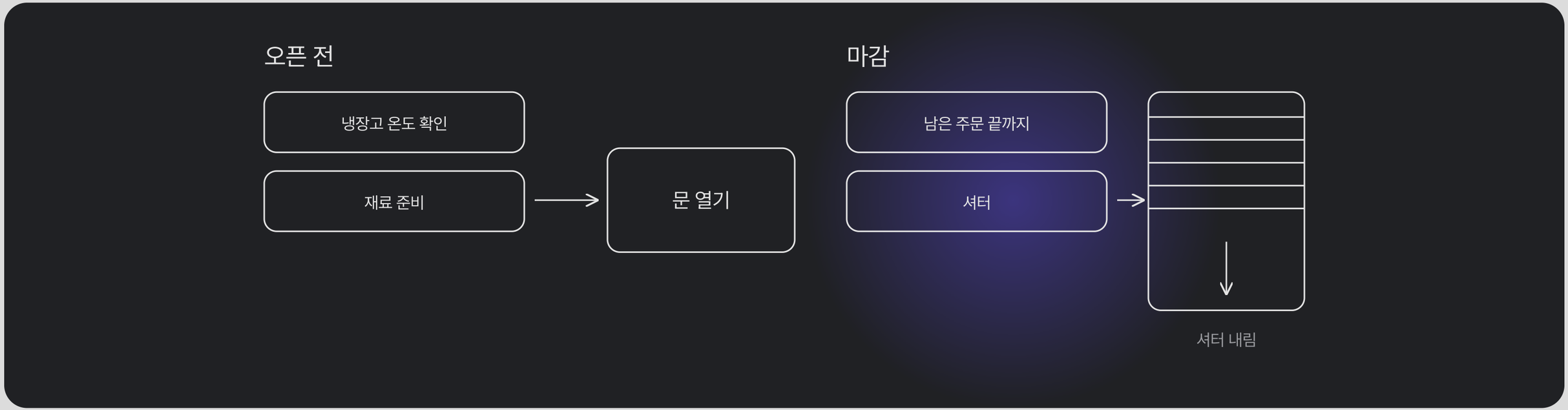
서버가 죽고 0.5초 뒤 결제사 승인

예매 기록은 없음

데이터 블록: WSL2 Ubuntu, Node 22.22.0, 서버 4029·가짜 외부 서버 5029, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

가게 오픈과 마감



→ 오픈 전: 냉장고 온도 확인, 재료 준비

→ 마감: 남은 주문을 끝까지 내고 셔터

→ 순서를 바꾸면: 고객이 들어온 뒤에 재료가 없음, 조리 중인 주문을 두고 셔터

© 한상훈 NestJS 백엔드 마스터

수명주기와 훅

→ 수명주기(lifecycle): 서버가 만들어지고, 요청을 받고, 내려가기까지의 단계

→ 훅(hook): 단계마다 Nest가 불러 주는 메서드

→ 프로바이더나 컨트롤러가 인터페이스를 구현하면 호출 대상

→ 뜰 때 둘, 내려갈 때 셋

뜰 때

```
onModuleInit
onApplicationBootstrap
```

내려갈 때

```
onModuleDestroy
beforeApplicationShutdown
onApplicationShutdown
```

© 한상훈 NestJS 백엔드 마스터

훅마다 로그 찍기

hook-logger.ts

```ts
export function hookLogger(name: string) {
  @Injectable()
  class HookLogger implements OnModuleInit, OnApplicationBootstrap,
    OnModuleDestroy, BeforeApplicationShutdown, OnApplicationShutdown {
    onModuleInit() { console.log(`${name} onModuleInit`); }
    onApplicationBootstrap() { console.log(`${name} onApplicationBootstrap`); }
    onModuleDestroy() { console.log(`${name} onModuleDestroy`); }
    beforeApplicationShutdown(signal?: string) { console.log(`${name} beforeApplicationShutdown(${signal})`); }
    onApplicationShutdown(signal?: string) { console.log(`${name} onApplicationShutdown(${signal})`); }
  }
  return HookLogger;
}
```

→ AppModule, ReservationsModule, NotificationsModule의 `providers`에 하나씩

→ `main.ts`: `listen` 앞뒤에 '호출', '끝' 로그 (시연용, 프로젝트 코드에는 없음)

© 한상훈 NestJS 백엔드 마스터

부팅 순서

서버 로그

```
ConcertsService 생성
SeatsService 생성
[InstanceLoader] ... dependencies initialized
listen 호출
NotificationsModule onModuleInit
[SmsOutbox] 발신번호 1588-1234
알림 채널: email, push, sms
ReservationsModule onModuleInit
AppModule onModuleInit
NotificationsModule onApplicationBootstrap
ReservationsModule onApplicationBootstrap
AppModule onApplicationBootstrap
[NestApplication] Nest application successfully started
listen 끝
```

→ 생성자 전부, 그다음 훅

→ onModuleInit: 가져다 쓰는 모듈부터
(Notifications → Reservations → App)

© 한상훈 NestJS 백엔드 마스터

종료 순서

서버 로그

```
$ kill -TERM <서버 PID>
AppModule onModuleDestroy
ReservationsModule onModuleDestroy
NotificationsModule onModuleDestroy
AppModule beforeApplicationShutdown(SIGTERM)
ReservationsModule beforeApplicationShutdown(SIGTERM)
NotificationsModule beforeApplicationShutdown(SIGTERM)
AppModule onApplicationShutdown(SIGTERM)
ReservationsModule onApplicationShutdown(SIGTERM)
NotificationsModule onApplicationShutdown(SIGTERM)
```

→ 모듈 순서는 부팅의 반대

→ 종료 쪽 두 훅은 받은 신호 이름을 인자로

© 한상훈 NestJS 백엔드 마스터

훅 다섯 개

훅	불리는 때
`onModuleInit`	그 모듈의 의존성이 다 만들어진 뒤
`onApplicationBootstrap`	모든 모듈의 onModuleInit이 끝난 뒤, 연결을 받기 전
`onModuleDestroy`	종료 신호를 받은 직후
`beforeApplicationShutdown`	모든 onModuleDestroy가 끝난 뒤, 연결을 닫기 전
`onApplicationShutdown`	연결이 다 닫힌 뒤

→ async로 쓰면 Nest가 끝날 때까지 기다림

출처: NestJS 공식 문서 Lifecycle events

© 한상훈 NestJS 백엔드 마스터

생성자에서 받아 오면: 발신번호 없음

요청 쪽 기록, 서버 로그, 발송사 로그

```
14:50:17.541 서버 응답 시작
14:50:17.542 R1 요청 보냄  → 201
[Nest] 2:50:17 PM  LOG [NestApplication] Nest application successfully started
[Nest] 2:50:18 PM  LOG [SmsOutbox] 발신번호 1588-1234
[Nest] 2:50:27 PM  LOG [SmsOutbox] 문자 1건 보냄: 400
14:50:27.285 [문자] 거절 1건: 발신번호 없음
```

→ 발신번호를 받기 전에 들어온 예매: from이 빈 채로 큐에

→ 발신번호 응답은 1.5초 걸리게 한 가짜 발송사

© 한상훈 NestJS 백엔드 마스터

생성자에서 받아 오면: 발송사가 꺼져 있을 때

서버 로그

```
[Nest]  LOG [InstanceLoader] SeatsModule dependencies initialized
알림 채널: email, push, sms
[Nest]  LOG [NestApplication] Nest application successfully started
TypeError: fetch failed
    at async SmsOutbox.loadSender (...)
  [cause]: Error: connect ECONNREFUSED 127.0.0.1:5029
Node.js v22.22.0
```

→ "started" 다음 줄에서 프로세스 종료

→ 요청 처리 중이 아닌 곳의 실패: 예외 필터가 받지 않음

© 한상훈 NestJS 백엔드 마스터

onModuleInit으로 옮긴 SmsOutbox

sms-outbox.ts

```ts
@Injectable()
export class SmsOutbox implements OnModuleInit, OnApplicationBootstrap {
  private timer: NodeJS.Timeout;

  async onModuleInit() {
    const res = await fetch(`${this.baseUrl}/sender`);
    ({ from: this.from } = await res.json());
    this.logger.log(`발신번호 ${this.from}`);
  }

  onApplicationBootstrap() {
    this.timer = setInterval(() => this.flush(), 10_000);
  }
}
```

→ 생성자: 비움 (객체를 만드는 일만)

© 한상훈 NestJS 백엔드 마스터

Nest가 기다린 부팅, 멈춘 부팅

발송사가 켜져 있을 때

```
[Nest] 3:11:54 PM  LOG [InstanceLoader] SeatsModule dependencies initialized
[Nest] 3:11:58 PM  LOG [SmsOutbox] 발신번호 1588-1234
[Nest] 3:11:58 PM  LOG [NestApplication] Nest application successfully started
15:12:10.281 [문자] 발송 1건
    1588-1234 → 010-1111-2222 "A-3-15 예매가 확정되었습니다"
```

발송사가 꺼져 있을 때

```
[Nest]  LOG [InstanceLoader] SeatsModule dependencies initialized
TypeError: fetch failed
    at async SmsOutbox.onModuleInit (...)
    at async NestApplication.init (...)
    at async NestApplication.listen (...)
  [cause]: Error: connect ECONNREFUSED 127.0.0.1:5029
```

→ 발송사가 꺼져 있으면: "started" 없음, 포트 안 열림

© 한상훈 NestJS 백엔드 마스터

타이머는 onApplicationBootstrap에서

`onModuleInit`

내 모듈이 쓸 것을 준비

발신번호

`onApplicationBootstrap`

모든 모듈 준비 뒤 시작할 일

10초 타이머

생성자

의존성을 받아 필드에 두는 일까지

© 한상훈 NestJS 백엔드 마스터

종료 신호

→ 시그널(signal): 운영체제가 프로세스에 보내는 짧은 알림

→ `SIGTERM`: 정리하고 끝내라는 요청, 배포 도구·docker stop·쿠버네티스가 보냄

→ `SIGINT`: 터미널의 Ctrl+C

→ `SIGKILL`: 즉시 종료, 프로세스가 막거나 처리할 수 없음

→ 처리 코드가 없으면 SIGTERM도 즉시 종료

© 한상훈 NestJS 백엔드 마스터

enableShutdownHooks 없이 받은 SIGTERM

서버 로그와 요청 쪽 기록

```
AppModule onApplicationBootstrap
[Nest]  LOG [NestApplication] Nest application successfully started
listen 끝
14:54:51.697 kill -TERM 서버
14:54:51.703 서버 종료
```

→ 종료 쪽 훅 로그 없음 (onModuleDestroy, beforeApplicationShutdown, onApplicationShutdown)

→ 뜰 때 훅은 enableShutdownHooks와 상관없이 불림

© 한상훈 NestJS 백엔드 마스터

enableShutdownHooks

main.ts

```ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.enableShutdownHooks();
  await app.listen(process.env.PORT ?? 3000);
}
```

→ 종료 신호를 받으면 종료 쪽 훅 세 개를 차례로 부름

→ 훅이 다 끝나면 같은 신호로 프로세스를 끝냄

→ 켠 쪽의 로그: 앞의 "종료 순서"

© 한상훈 NestJS 백엔드 마스터

기본이 꺼져 있는 이유

→ 신호마다 `process`에 리스너를 붙임 (메모리 사용)

→ 한 프로세스에 Nest 앱 여러 개 (병렬 테스트 등): 리스너가 많다는 Node 경고

→ Windows: SIGTERM이 오지 않음, 작업 관리자 종료는 무조건 종료 (SIGINT·SIGBREAK는 동작)

출처: NestJS 공식 문서 Lifecycle events

© 한상훈 NestJS 백엔드 마스터

남은 문자를 onModuleDestroy에서

sms-outbox.ts

```
@Injectable()
export class SmsOutbox implements OnModuleInit, OnApplicationBootstrap, OnModuleDestroy {
  async onModuleDestroy() {
    clearInterval(this.timer);
    await this.flush();
    this.logger.log('onModuleDestroy: 남은 문자 보냄');
  }
}
```

→ main.ts: app.enableShutdownHooks() 켬

→ 배포 순간 시연을 다시: 결제사 2.5초, 예매 두 건, SIGTERM, 0.3초 뒤 R3

© 한상훈 NestJS 백엔드 마스터

처리 중이던 예매의 문자

요청 쪽 기록

```
15:12:53.005 R1 요청 보냄
15:12:55.007 R2 요청 보냄
15:12:55.768 R1 끝 [201]
15:12:57.009 kill -TERM 서버
15:12:57.347 R3 끝  curl: (7) Failed to connect to localhost port 4029
15:12:57.543 R2 끝 [201]
  {"id":2,"seatId":"A-3-16",...,"message":"[문자 → 010-3333-4444] A-3-16 예매가 확정되었습니다"}
```

서버 로그

```
[SmsOutbox] 문자 1건 보냄: 200
[SmsOutbox] onModuleDestroy: 남은 문자 보냄
[예매] A-3-16 결제 완료
```

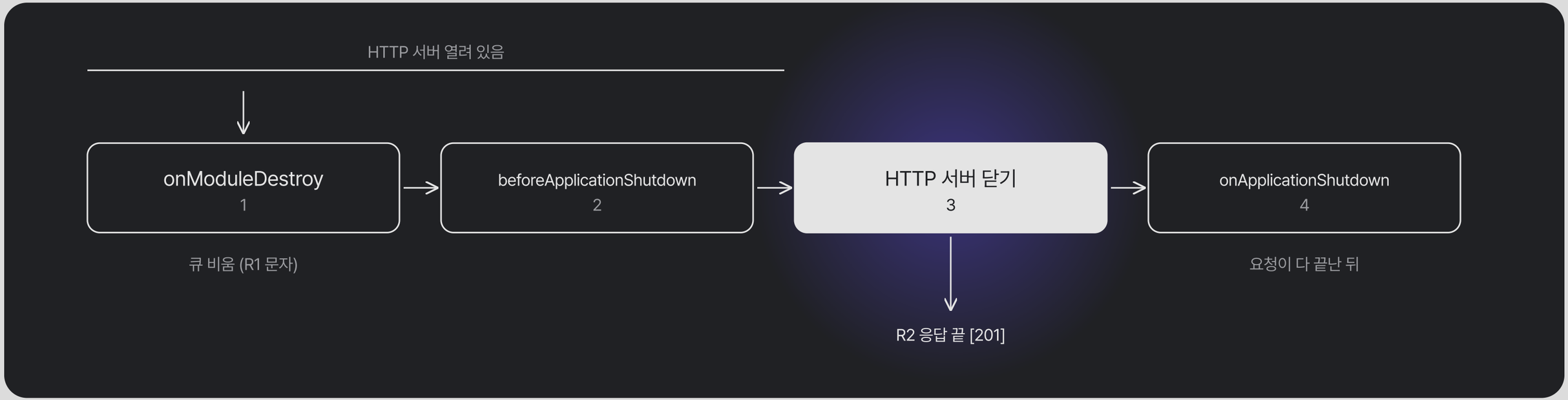
발송사 로그

```
15:12:57.011 [문자] 발송 1건
    1588-1234 → 010-1111-2222 "A-3-15 예매가 확정되었습니다"
```

→ R2: 201과 확정 메시지, 문자는 발송 기록 없음

© 한상훈 NestJS 백엔드 마스터

종료 순서와 HTTP 서버



→ 1 `onModuleDestroy`: HTTP 서버는 아직 열려 있음

→ 2 `beforeApplicationShutdown`: 아직 열려 있음

→ 3 HTTP 서버 닫기: 새 연결 거절, 처리 중인 요청이 끝날 때까지 기다림

→ 4 `onApplicationShutdown`: 처리 중이던 요청까지 다 끝난 뒤

© 한상훈 NestJS 백엔드 마스터

남은 문자를 onApplicationShutdown에서

sms-outbox.ts

```
@Injectable()
export class SmsOutbox implements OnModuleInit, OnApplicationBootstrap, OnApplicationShutdown {
  async onApplicationShutdown(signal?: string) {
    clearInterval(this.timer);
    await this.flush();
    this.logger.log(`onApplicationShutdown(${signal}): 남은 문자 보냄`);
  }
}
```

© 한상훈 NestJS 백엔드 마스터

두 건 다 나간 종료

요청 쪽 기록

```
15:13:38.153 R1 요청 보냄
15:13:40.072 R2 요청 보냄
15:13:40.816 R1 끝 [201]
15:13:42.075 kill -TERM 서버
15:13:42.406 R3 끝  curl: (7) Failed to connect to localhost port 4029
15:13:42.589 R2 끝 [201]
```

서버 로그

```
[예매] A-3-15 결제 완료
[예매] A-3-16 결제 완료
[SmsOutbox] 문자 2건 보냄: 200
[SmsOutbox] onApplicationShutdown(SIGTERM): 남은 문자 보냄
```

가짜 외부 서버 로그

```
15:13:42.583 [결제사] 승인 pg-60000
15:13:42.587 [문자] 발송 2건
    1588-1234 → 010-1111-2222 "A-3-15 예매가 확정되었습니다"
    1588-1234 → 010-3333-4444 "A-3-16 예매가 확정되었습니다"
```

© 한상훈 NestJS 백엔드 마스터

우아한 종료

→ 우아한 종료(graceful shutdown): 하던 일을 마무리하고 내려가는 종료

→ 기다리는 시간에도 상한: 유예 시간이 지나면 SIGKILL

1

새 요청을 멈춤

2

처리 중인 요청을 끝냄

3

자원을 닫음

큐 비우기, 타이머, 커넥션 풀

4

프로세스 종료

데이터 블록: docker stop 기본 10초, 쿠버네티스 파드 기본 30초 (부록), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

보내기 실패하면

sms-outbox.ts

```
async flush() {
  if (this.queue.length === 0) return;
  const batch = this.queue.splice(0);
  try {
    const res = await fetch(`${this.baseUrl}/send`, {
      method: 'POST',
      body: JSON.stringify(batch),
    });
    this.logger.log(`문자 ${batch.length}건 보냄: ${res.status}`);
  } catch (err) {
    this.queue.unshift(...batch);
    this.logger.error(`문자 ${batch.length}건 못 보냄: ${(err as Error).message}`);
  }
}
```

→ 타이머 안의 거부가 프로세스를 끝내지 않게

→ 실패한 묶음은 큐 앞에 되돌림, 다음 주기에 다시

© 한상훈 NestJS 백엔드 마스터

훅마다 맡길 일

훅	맡길 일	이 프로젝트
생성자	의존성 받기	서비스 필드
`onModuleInit`	내 모듈이 쓸 것 준비, 실패하면 부팅 중단	발신번호 받기, 알림 채널 모으기
`onApplicationBootstrap`	모든 모듈 준비 뒤 시작할 일	10초 발송 타이머
`onModuleDestroy`, `beforeApplicationShutdown`	요청이 아직 도는 동안의 준비	(없음)
`onApplicationShutdown`	요청이 다 끝난 뒤 자원 닫기	타이머 끄기, 남은 문자 보내기

→ `main.ts`: `app.enableShutdownHooks()`

© 한상훈 NestJS 백엔드 마스터

종료의 순서

"종료 신호를 받으면 새 요청을 멈추고, 처리 중인 요청을 끝낸 뒤에 자원을 닫습니다."